

Project Title: Automating Employee Data

Technology: MERN Stack (MongoDB, Express.js, React, Node.js)

Team Members:

- PRITHIKA S – Team Lead
- MUTHUMARI M – Member
- ANFIYAA M – Member
- VISHALINI K – Member

Introduction

- Purpose: Automate employee data management to reduce errors and manual effort.
- Scope: Manage employee records, attendance, payroll, and performance data efficiently.
- Benefit: Streamlined operations, improved accuracy, and real-time insights.

Project Overview

Purpose:

- Provide a centralized, secure system for employee data.
- Enable real-time analytics for HR decision-making.

Key Features:

- Employee record management
- Attendance tracking
- Payroll automation
- Dashboard with analytics
- Secure login and role-based access

Architecture Overview

- **Frontend (React):** Component-based structure, interactive UI, API communication via Axios.
- **Backend (Node.js + Express.js):** RESTful APIs, business logic, middleware for authentication and validation.
- **Database (MongoDB):** Stores employee data, flexible schema, uses Mongoose for data modeling.

Frontend Architecture

- React.js with component-based architecture
- Responsive design using CSS/Bootstrap
- Pages include Dashboard, Employee List, Attendance, Payroll
- Axios handles API requests to the backend

Backend Architecture

- Node.js with Express.js framework
- Routes handle CRUD operations
- Controllers manage business logic
- Middleware for JWT authentication and error handling

Database Schema

- MongoDB NoSQL database
- Collections: Employees, Attendance, Payroll, Users
- Each collection contains relevant fields (e.g., Employee: name, email, role, department)
- Mongoose used for schema modeling and validation

Setup Instructions

Prerequisites:

- Node.js (v14+)
- MongoDB (local or cloud)
- npm/yarn

Installation Steps:

1. Clone repository: `git clone <repo-url>`
2. Navigate to server folder: `npm install`
3. Navigate to client folder: `npm install`
4. Configure `.env` with database URI, JWT secret, and port

Folder Structure

Client (React):

- `/src/components` – UI components
- `/src/pages` – Dashboard, Employee List, Attendance, Payroll
- `/src/services` – API service functions

Server (Node.js):

- `/routes` – API endpoints
- `/models` – Mongoose schemas
- `/controllers` – Business logic
- `/middleware` – Authentication, error handling

Running the Application

Frontend:

- Navigate to `client` folder
- Run: `npm start`

Backend:

- Navigate to server folder
- Run: `npm start`

Access:

- Frontend available at <http://localhost:3000>
- Backend API available at <http://localhost:5000>

API Documentation

Endpoints:

- GET `/empResponse`:
[
 { "id": "1", "name": "John Doe", "email":
 "john@example.com" }
]
•
• `/employees` – Fetch all employees
• POST `/employees` – Add a new employee
• PUT `/employees/:id` – Update employee data
• DELETE `/employees/:id` – Remove employee

Request/Response Example:

GET `/employees`

Authentication & Authorization

- JWT-based authentication
- Protected routes for admin and HR roles
- Tokens stored in `localStorage` (frontend)
- Middleware validates tokens and role-based access

User Interface

- Dashboard: Displays employee analytics
- Employee List: CRUD operations
- Attendance & Payroll pages
- Responsive UI with tables and charts
- Screenshots/GIFs to demonstrate interactions

Testing & Demo

Testing Strategy:

- Unit testing with Jest
- API testing with Postman
- Manual UI testing for responsiveness

Demo:

- Provide link to live demo or screenshots of key features

Known Issues & Future Enhancements

Known Issues:

- Some UI responsiveness issues on mobile
- Minor bugs in payroll calculations

Future Enhancements:

- AI-based employee analytics
- Mobile app version
- Cloud deployment
- Integration with third-party HR tools

Detailed Employee Data Model

Employee Collection Fields:

- `_id`: Unique identifier
- `name`: Employee full name
- `email`: Contact email
- `role`: Admin / HR / Employee
- `department`: Department name
- `joiningDate`: Date of joining
- `salary`: Monthly salary
- `attendance`: Array of attendance records

Attendance Collection Fields:

- `employeeId`: Reference to Employee `_id`
- `date`: Attendance date
- `status`: Present / Absent / Leave

Payroll Collection Fields:

- `employeeId`
- `basicSalary`
- `deductions`
- `bonus`
- `NetSalary`

API Design – Sample Requests

Add Employee:

POST /employees

Body:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "role": "Employee",
  "department": "IT",
  "salary": 50000
}
```

Response:

```
{ "success": true, "employeeId": "12345" }
```

Get All Employees:

GET /employees

Response:

```
[  
  { "name": "John Doe", "email": "john@example.com", "role":  
    "Employee" }  
]
```

React Routing

- React Router manages navigation:
 - /login → Login page
 - /dashboard → Admin/HR dashboard
 - /employees → Employee management page
 - /attendance → Attendance tracking
 - /payroll → Payroll management

Testing Strategy

- **Unit Testing:** Jest for React components and Node.js functions
- **API Testing:** Postman or Insomnia for CRUD APIs
- **End-to-End Testing:** Cypress for UI flow simulation
- **Test Coverage Reports:** Ensure critical functions are tested